Михаил Разаков

C++ / Systems Engineer — LLVM · Compiler Tooling

Москва / удалённо · razakov.mikhail@outlook.com · Telegram · GitHub

ПРОФИЛЬ

Разрабатываю C++/LLVM-инструменты для анализа программ и консервативных преобразований IR. В МЦСТ работал над экспериментальным LLVM LICM, сейчас занимаюсь символьными вычислениями в SharpChecker (ИСП РАН). В собственных проектах реализую анализ зависимостей памяти (GNU C17), LLVM Global IV (C++23) и x86-64 backend на Rust. Проверяю результаты регрессионными и дифференциальными тестами.

ТЕХНОЛОГИИ

C++ и инструменты: C++23, GNU C17 (POSIX), LLVM 22/IR, CMake, GCC/Clang, Git, Linux, Python

Алгоритмы и компиляторы: CFG/SSA, dominance, data-flow, межпроцедурные эффекты, memory dependence; графовые алгоритмы, Rust/x86-64 SysV ABI

Проверки: Регрессионные и дифференциальные тесты, эталонные оракулы, ASan/UBSan, анализ контрпримеров, консервативная проверка допустимости

ОПЫТ

ИСП РАН · SharpChecker · октябрь 2026 — настоящее время

• Работаю над символьными вычислениями для статического анализа программ на C#/.NET.

МЦСТ · стажировка по разработке компиляторов · июль — август 2026

• Разрабатывал экспериментальный LLVM LICM с проверкой структуры цикла, обращений к памяти и допустимости переноса инструкций; сравнивал поведение до/после на Python.

• Реализовал на C++23 алгоритмы RPO, Dijkstra, Dinic и Tarjan SCC; локальный прогон из Git-снимка прошёл 14/14 fixture-тестов.

ИНЖЕНЕРНЫЕ ПРОЕКТЫ

LLVM Interprocedural Global IV Optimization

LLVM 22 pass локализует поддерживаемые обращения к целочисленным глобальным переменным в циклах. Анализ эффектов и отдельная PromotionLegality предшествуют IR-преобразованию. На зафиксированном LLVM 22.1.8 пройдены 29/29 регрессионных сценариев с положительными и консервативно отклонёнными преобразованиями.

PS-form Memory Dependence Analyzer

GNU C17-анализатор пересечений памяти между разными итерациями. Точный подсчёт для поддерживаемых аффинных случаев, модульные/НОД-проверки, ограниченный перебор. В неразрешённых случаях возвращает консервативное maybe; проект выполнен как техническое задание отбора МЦСТ, не промышленный анализатор.

x86-64 Compiler Backend & Register Allocation

На Rust реализован экспериментальный backend для ограниченного IR: SSA/CFG, liveness, register allocation, spills, phi lowering и SysV x86-64. Независимый verifier проверяет назначения регистров/stack slots, а результаты native-кода сравниваются с эталонным IR-интерпретатором.

ОБРАЗОВАНИЕ

НИУ ВШЭ — Программная инженерия

ДОСТИЖЕНИЯ

2026 · Отбор МЦСТ — LLVM/C++: По письму организатора: единственная оценка 5,0/5,0 и единственное решение с 104/104 официальных тестов среди 49 работ.

2025–2026 · «Юниор» НИЯУ МИФИ: Дипломы I степени в секции информационных технологий за 2025 и 2026 годы.

2026 · LangDev — UniversalToolchain: Доклад 9 октября 2026 года; опубликована видеозапись организаторов. — Запись доклада